Class 6: R Functions

Zeyu Qi(A17342618)

Table of contents

Background

All functions in R have at least 3 things:

- a *name* (we pick it as use it to call the function)
- input *arguments* (one or more comma seperated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

Q1. A first function

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can be either "required" or "optional". The latter one would have a default value.

> Q1a. Your first version of the function should add two input numbers together.

```
add <- function(x, y=1000) {
  x + y
}
```

Can we use our new function:

```
add(10,100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 1010
```

> Q1b: For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10)).

```
add <- function(x, y=0) {
  sum(x + y)
}
```

```
add(4,7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

> Q1c: Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return 2. Hint: explore the … (dots) argument or the base R function sum().

```
add <- function(...) {
  sum(...)
}
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a fucntion (rather than the default last line) by using `return()` function call.

```
add <- function(x, y=0, z=0) {
  sum(x,y,z)
  cat("Is it break time yet?\n")
}
```

```
add(100, 10)
```

```
Is it break time yet?
```

Q2. Write a generate_dna() function

A useful function here is the "base R' `sample()` function:

```
sample(1:5, size=1)
```

```
[1] 4
```

```
sample(1:5, size=6, replace = TRUE)
```

```
[1] 2 3 5 4 2 4
```

We can use this to make a random nucleotide sequence if we drawn from "A", "C", "T", and "G".

```
sample(c("A", "C", "T", "G"), size=10, replace = TRUE)
```

```
 [1] "T" "T" "T" "G" "C" "C" "T" "C" "T" "C"
```

> Q2a. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return "A", "T", "T", "G", "A", "C".

```
generate_dna <- function(len=10) {
  sample(c("A", "C", "T", "G"), size=len, replace = TRUE)
}
```

```
generate_dna(6)
```

```
[1] "T" "T" "A" "G" "C" "T"
```

Q2b. Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a singe vector of multiple letters). For example "AAGGCTG".

```
generate_dna <- function(len=10, single.element=TRUE) {
  ans <- sample(c("A", "C", "T", "G"), size=len, replace = TRUE)
  if (single.element) {
    ans <- paste(ans, collapse = "")
  }
  return(ans)
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

```
generate_dna(6, TRUE)
```

```
[1] "ATTGTC"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
>len9
CGAAGGCTG
```

```
generate_dna <- function(len=10, single.element=TRUE) {
  ans <- sample(c("A", "C", "T", "G"), size=len, replace = TRUE)
  if (single.element) {
    ans <- paste(ans, collapse = "")
  }

  cat(paste(">len", len, "\n", sep=""))
  cat(ans)
}
```

```
generate_dna(9)
```

```
>len9
CAAATGCAC
```

Q3. Write a generate_protein() function

> Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG".

```
generate_protein <- function(len=10, single.element=TRUE) {

  ans <- sample(c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P",
"T", "W", "Y", "V"), size=len, replace = TRUE)
  if (single.element) {
    ans <- paste(ans, collapse = "")
  }
  return(ans)
}
```

```
generate_protein(6)
```

```
[1] "AWSKHV"
```

Q4. Generate random protein sequences of length 6 to 13

```
generate_protein <- function(len=10, single.element=TRUE) {
  for (len in 6:13) {
    ans <- sample(c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P"
"T", "W", "Y", "V"), size=len, replace = TRUE)
  if (single.element) {
    ans <- paste(ans, collapse = "")
  }

  cat(paste(">id.", len, "\n", sep=""))
  cat(ans)
  cat("\n")
```

```
  }
}
```

```
generate_protein()
```

```
>id.6
RVYPVN
>id.7
QKWNFDM
>id.8
CYCNSGNR
>id.9
CNIDVICAH
>id.10
CESPMMSYYD
>id.11
VEEMKATWSWS
>id.12
ENQTDHVCIRDD
>id.13
QVNIELHIPPSAM
```

Q5. BLASTp search against nr — are your peptides "unique in nature"?

Q5. Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at https://blast.ncbi.nlm.nih.gov/. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
6	100 %	100 %	N
7	100 %	100 %	N
8	100 %	100 %	N
9	88.89 %	100 %	Y
10	90.00 %	100 %	Y
11	81.82 %	91 %	Y
12	83.33 %	83 %	Y
13	88.89 %	69 %	Y

Q5a. At which sequence length do your randomly generated peptides start to look "unique in nature" (i.e. no 100% coverage + 100% identity hit)?

At sequence length 9, the peptides start to look unique.

Q5b. peculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20 for a peptide of length) and to the size of the known protein universe.

Very short peptides are almost always found in nr because the sequence space (20^L) is relatively small for small L, so many possible sequences are already represented in the nature. As peptide length increases, the sequence space grows exponentially, quickly exceeding the total number of sequences that exist or have been sampled in nature.

Q6. Connecting your findings to immunology (MHC class II and T-cell activation)

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

The minimum length is likely around 8–10 amino acids or longer, since shorter peptides are extremely common in the protein universe. From Q5, very short peptides occur frequently because the sequence space (20^L) is small, meaning many different proteins—self and non-self—share the same short sequences. This would make it difficult for the immune system to reliably distinguish self from non-self, increasing the risk of false positives or autoimmunity. Longer peptides, with a much larger sequence space, are more likely to be unique and therefore better indicators of foreign proteins. Thus, presenting very short peptides would reduce specificity and make immune recognition less accurate.